

Contrôle de connaissance Correction

Unité GIN-SR2I202 – Cryptographie et PKI
Durée : 1h30

*Les supports de cours sont autorisés.
Les dispositifs électroniques sont interdits.
Les réponses peuvent être rédigées en anglais.
La rédaction et la justification des réponses seront prises en compte.*

Tout au long de ce devoir, nous allons nous intéresser au protocole IKEv2, qui permet à deux entités, appelées Initiateur (noté $\mathcal{I}$) et Receveur (noté $\mathcal{R}$), de s'authentifier mutuellement, de créer et partager une clé cryptographique de session, et de finalement s'échanger des données de session de façon confidentielle et intègre via cette clé partagée. Ce protocole est utilisé pour mettre en place les informations de sécurité partagées dans IPsec (base des communications protégées sur des réseaux IP).

1 Échange de clé

Dans IKEv2, l'échange de clé va être basé sur le protocole Diffie-Hellman.

Question 1 (4 points)

- Le protocole Diffie-Hellman de base est sujet à l'attaque de l'homme du milieu. Décrivez cette attaque et expliquez ce qu'il faut rajouter entre $\mathcal{I}$ et $\mathcal{R}$ pour l'empêcher.
- Déterminer la clé Diffie-Hellman mk commune à l'Initiateur et au Receveur dans le cas où nous utilisons le corps $(\mathbb{Z}_{23}, +, \times)$, où le générateur utilisé est $g = 3$ et en supposant que l'Initiateur $\mathcal{I}$ choisit un nombre secret $x = 6$ et le Receveur $\mathcal{R}$ choisit un nombre secret $y = 15$. Vous fournirez les détails de vos calculs dans la copie.

Correction.

- Un acteur malintentionné pourrait se mettre entre $\mathcal{I}$ et $\mathcal{R}$ en générant un z et en calculant g^z , puis en participant à deux protocoles Diffie-Hellman, l'un avec $\mathcal{I}$ et l'autre avec $\mathcal{R}$ pour finalement partager une clé avec chacun d'eux (g^{xz} avec $\mathcal{I}$ et g^{yz} avec $\mathcal{R}$), de façon totalement imperceptible par ces deux protagonistes.

Il manque la signature numérique des messages échangés par l'Initiateur et le Receveur, qui doit nécessairement s'appuyer sur une PKI pour que les clés de signature soient certifiées.

- L'Initiateur envoie au Receveur la valeur $g^x \pmod{p} = 3^6 \pmod{23} = 16$. En effet,

$$3^6 = 3 \pmod{23}$$

$$\begin{aligned} 3^2 &= 9 \pmod{23} \\ 3^4 &= 9^2 = 81 = 12 \pmod{23} \\ 3^8 &= 12^2 = 144 = 6 \pmod{23}. \end{aligned}$$

Il s'ensuit que $3^6 = 3^4 \cdot 3^2 = 12 \cdot 9 = 108 = 16 \pmod{23}$ Le Receveur envoie en retour la valeur $g^y \pmod{p} = 3^{15} \pmod{23} = 12$ car $3^{15} = 3^8 \cdot 3^4 \cdot 3^2 \cdot 3 = 6 \cdot 12 \cdot 9 \cdot 3 = 1944 = 12 \pmod{23}$. L'Initiateur peut maintenant calculer la clé secrète :

$$\text{mk} = (g^y \pmod{p})^x = 12^6 \pmod{23} = 9.$$

Eventuellement, nous pouvons vérifier que le Receveur calculera bien la bonne clé par :

$$\text{mk} = (g^x \pmod{p})^y = 16^{15} \pmod{23} = 9.$$

2 Génération des clés de session

Dans le protocole IKEv2, la clé Diffie-Hellman mk partagée par $\mathcal{I}$ et $\mathcal{R}$ est ensuite utilisée pour

1. protéger l'intégrité des échanges de l'authentification via un MAC ;
2. protéger l'intégrité et la confidentialité des données de session qui seront ensuite échangées.

Dans le protocole IKEv2, ce n'est pas la clé mk qui va directement être utilisée pour ces deux besoins. À la place, il va y avoir une étape de dérivation de clés de session à partir de mk . Pour cette dérivation, IKEv2 utilise une fonction pseudo-aléatoire PRF qui va s'appuyer sur une fonction de hachage $\mathcal{H}$ et la fonction f suivante :

$$f(m) := \mathcal{H}(\text{mk} \parallel \mathcal{H}(\text{mk} \parallel m)).$$

où $\parallel$ est l'opération de concaténation.

Le message m va prendre les valeurs successives $n_{\mathcal{I}} \parallel n_{\mathcal{R}} \parallel 01$ (pour générer la clé k_t utiliser pour vérifier l'intégrité des échanges de l'authentification), $n_{\mathcal{I}} \parallel n_{\mathcal{R}} \parallel 02$ (pour générer la clé k_s utilisée pour contrôler l'intégrité et la confidentialité des données futures), où $n_{\mathcal{I}}$ (resp. $n_{\mathcal{R}}$) est un aléa choisit par $\mathcal{I}$ (resp. $\mathcal{R}$) et envoyé en clair au moment de l'échange Diffie-Hellman.

Question 2 (4 points)

- Pourquoi ne peut-on pas utiliser directement la clé Diffie-Hellman mk pour l'ensemble de ces deux besoins ?
- Quelle fonction de hachage préconisez-vous d'utiliser ici ?
- A quoi vous fait penser la fonction f ?
- Que pensez-vous du fait que les messages m soient d'une part uniquement sur des données publiques, et d'autre part aussi peu différents les uns des autres ?

Correction.

- Parce que la clé mk sera trop utilisée, et il est toujours plus prudent de générer une clé différente pour chaque besoin.
- Nous pouvons utiliser SHA256, SHA512 ou SHA3.
- Il s'agit du mode HMAC.
- Cela ne pose aucun problème car le fait que la clé utilisée soit secrète suffit pour assurer la sécurité de l'ensemble (principes de Kerckhoffs). Cela permet même d'éviter à $\mathcal{I}$ et $\mathcal{R}$ de devoir s'échanger un secret supplémentaire qui serait basé sur une graine différente. Le fait que les données en entrées soit peu différentes ne pose aussi aucun problème, pour les mêmes raisons.

3 Authentification des échanges

L'authentification des parties pendant l'échange de clé va s'appuyer d'une part sur une signature numérique (notée σ), et d'autre part sur un MAC (noté τ , en utilisant la clé k_t précédemment générée).

Question 3 (4 points)

- En vous appuyant sur l'ensemble des questions précédentes, déterminez quelles éléments doivent obligatoirement être signés par $\mathcal{I}$ et $\mathcal{R}$ pour assurer la sécurité de leurs échanges. Expliquez votre réponse.
- Un Initiateur $\mathcal{I}$ particulier vient de perdre sa clé privée de signature. Peut-il encore signer des messages qu'il envoie à $\mathcal{R}$? Peut-il encore vérifier des messages signés qu'il reçoit de $\mathcal{R}$? A quoi peut encore servir la clé publique de $\mathcal{I}$?

Correction.

- Il faut que l'ensemble des éléments échangés et servant à générer les clés k_t et k_s doivent être authentifiés, à savoir g^x (resp. g^y pour $\mathcal{R}$), mais aussi les aléas $n_{\mathcal{I}}$ et $n_{\mathcal{R}}$.
- Non, il ne peut plus signer avec cette clé. Il doit obligatoirement en changer. Oui, il peut toujours vérifier les messages qu'il reçoit de $\mathcal{R}$ puisque la clé de ce dernier n'a pas été perdue. La clé publique de $\mathcal{I}$ ne sert plus à rien, excepté peut-être vérifier des signatures précédemment générées par $\mathcal{I}$, selon le contexte.

4 Choix de la suite cryptographique

Une fois l'authentification mutuelle effectuée, l'Initiateur et le Receveur vont pouvoir s'échanger des données en les sécurisant via la clé k_s générée et partagée de façon authentifiée. Pour cela, il faut utiliser un algorithme de contrôle d'intégrité (un MAC) et un algorithme de chiffrement (noté ENC).

Dans IKEv2, l'un des algorithmes proposés correspond à Camellia en mode CCM. La structure de Camellia est fournie en Figure 1, où les clés k_i , k_l et kw_j sont dérivées de la clé k_s .

Question 4 (4 points)

- Quelle(s) structure(s) connue(s) reconnaissez-vous dans le schéma Camellia ? A quel(s) endroit(s) ?
- Expliquez avec un maximum de détails en quoi vous retrouvez dans Camellia les principes fondamentaux introduits par Shannon pour les systèmes de chiffrement par blocs.
- Le mode CCM (counter with cipher block chaining message authentication code) combine le mode CTR avec le mode CBC, utilisés en version "MAC-then-Encrypt". Faites le diagramme d'un tel mode.

Correction.

- On reconnaît un schéma de Feistel dans les blocs de 6 tours mais aussi dans la fonction F (de façon un peu généralisée).
- Dans la fonction F , nous reconnaissons une xor avec les clés k_i , puis une fonction de substitution, et enfin une permutation. Nous retrouvons le fait d'effectuer ses fonctions plusieurs fois grâce aux tours effectués. Le schéma de Feistel à 6 tours utilisé plusieurs fois rajoute aussi à la diffusion via des permutations.
- On prend le message m à envoyer, on fait un CBC-MAC avec Camellia sur ce dernier pour obtenir τ . Ensuite, on concatène m et τ et on envoie le tout dans le mode CTR utilisant Camellia.

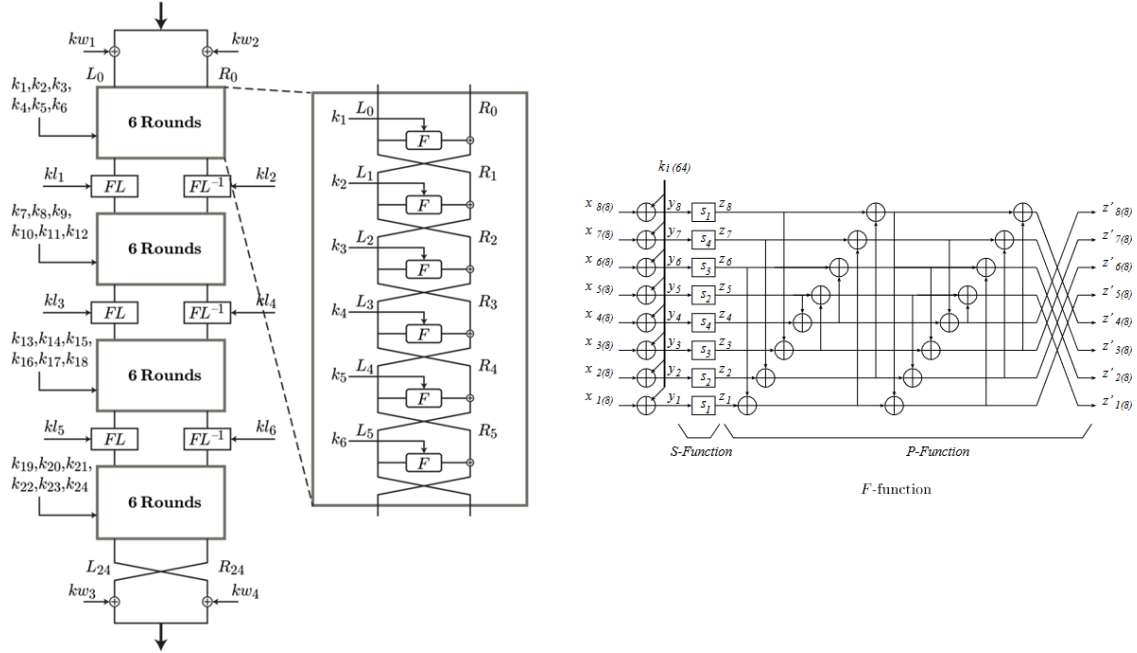


Figure 1: Structure de Camellia

5 Mise en œuvre

Une société Lambda veut mettre en œuvre le protocole IKEv2 pour un nouveau produit qu'elle souhaite mettre sur le marché en Europe. Dans son cas, les calculs cryptographiques du Receveur seront effectués par un composant hardware qu'elle a acheté auprès d'une société Gamma en Italie qui a de bonnes critiques. Pour l'Initiateur, le travail a été confié à un stagiaire de Lambda pour une implémentation software. Pour la génération du secret x Diffie-Hellman de l'Initiateur, le stagiaire a fait le choix de générer un nombre aléatoire et de l'incrémenter à chaque nouvelle iteration du protocole. Pour la suite cryptographique, l'équipe cryptographique de la société Lambda a décidé de prendre AES-128 mais pour un soucis de rapidité, de ne faire que 9 tours au lieu de 10. Enfin, le mode opératoire utilisé choisi est le mode CBC, utilisé deux fois en parallèle avec le message à envoyer, une fois pour la confidentialité et une autre fois pour l'intégrité.

Question 5 (4 points)

- Pointez du doigt tout ce qui ne va pas dans les choix de cette société et expliquez en quelques mots ce qu'elle aurait dû faire à la place.
- La société Lambda décide d'utiliser RSA comme schéma de signature numérique. Pour simplifier un peu, ils décident que l'Initiateur et le Receveur vont utiliser le même module RSA n , mais des exposants public et secret différents. Nous noterons (e_I, d_I) les exposants public et secret de I et (e_R, d_R) les exposants public et secret de R . En utilisant les propriétés des clés RSA, montrez comment par exemple l'Initiateur peut se faire passer pour le Receveur en étant capable de créer une clé privée d'équivalente de d_R , sachant qu'il connaît le module n , la clé publique e_R de R , et sa propre paire de clés (e_I, d_I) .

Correction.

- Confier le développement à un unique stagiaire. La génération de l'aléa x n'est pas bon car il ne faut

partir d'un aléa et faire une simple incrémentation. Si l'un est corrompu, l'ensemble le sera. Il faut nécessairement conserver AES en 10 tours, et ne jamais modifier un standard. Le mode CBC n'est pas à utiliser pour assurer la confidentialité. Et le mode "Encrypt-and-MAC" n'est pas une façon idéale de procéder.

- L'Initiateur connaît n , la clé publique $e_{\mathcal{R}}$ de $\mathcal{R}$, et sa propre paire de clés $(e_{\mathcal{I}}, d_{\mathcal{I}})$. Soit k le pgcd de $e_{\mathcal{R}}$ et de $e_{\mathcal{I}}d_{\mathcal{I}} - 1$. Par l'algorithme d'Euclide étendu, il est possible de trouver u et v tels que

$$u \cdot e_{\mathcal{R}} + v \cdot (e_{\mathcal{I}}d_{\mathcal{I}} - 1) = k.$$

Or, en réduisant cette égalité modulo $\varphi(n)$, et en remarquant que $e_{\mathcal{I}}d_{\mathcal{I}} = 1 \pmod{\varphi(n)}$ (et donc que $e_{\mathcal{I}}d_{\mathcal{I}} - 1 = 0 \pmod{\varphi(n)}$) nous obtenons

$$\begin{aligned} u \cdot e_{\mathcal{R}} &= k \pmod{\varphi(n)} \\ (uk^{-1}) \cdot e_{\mathcal{R}} &= 1 \pmod{\varphi(n)} \end{aligned}$$

$\mathcal{I}$ peut inverser k puisqu'elle connaît $\varphi(n)$.

Nous pouvons en conclure que ud^{-1} est donc un exposant équivalent à $d_{\mathcal{R}}$ et peut donc permettre à $\mathcal{I}$ de signer n'importe quel message de son choix en se faisant passer pour $\mathcal{R}$.